

WESTERN SYDNEY UNIVERSITY



SCHOOL OF Computer Engineering Data and Math

GROUP ASSIGNMENT COVER SHEET

STUDENT DETAILS

Student name:	<u>Muhammed Mafawalla(Group Leader)</u>	Student ID number:	<u>22024540</u>
Student name:	<u>Manan Bhatia</u>	Student ID number:	<u>22035587</u>
Student name:	<u>Rosmit Gyawali</u>	Student ID number:	<u>22069030</u>
Student name:	<u>Shekhar Patel</u>	Student ID number:	<u>22034335</u>
Student name:	<u></u>	Student ID number:	<u></u>

UNIT AND TUTORIAL DETAILS

Unit name:	<u>Social Web Analytics</u>	Unit number:	<u>COMP3020</u>
Tutorial/Lecture :	<u>Tuesday 3 to 5 pm</u>	Class day and time:	<u>Friday, 11am to 1pm</u>
Lecturer or Tutor name:	<u>Gizem Intepe</u>		

ASSIGNMENT DETAILS

Title:	<u>Group Project</u>				
Length:	<u></u>	Due date:	<u>20 October 2023</u>	Date submitted:	<u>20 October 2023</u>
Home campus (where you are enrolled):	<u>Parramatta South</u>				

DECLARATION

- ☒ I hold a copy of this assignment if the original is lost or damaged.
- ☒ I hereby certify that no part of this assignment or product has been copied from any other student's work or from any other source except where due acknowledgement is made in the assignment.
- ☒ I hereby certify that no part of this assignment or product has been submitted by me in another (previous or current) assessment, except where appropriately referenced, and with prior permission from the Lecturer / Tutor / Unit Coordinator for this unit.
- ☒ No part of the assignment/product has been written/produced for me by any other person except where collaboration has been authorised by the Lecturer / Tutor /Unit Coordinator concerned.
- ☒ I am aware that this work will be reproduced and submitted to plagiarism detection software programs for the purpose of detecting possible plagiarism **(which may retain a copy on its database for future plagiarism checking).**

Student's signature:	<u>M.Mafawalla</u>
Student's signature:	<u>M.Bhatia</u>
Student's signature:	<u>R.Gyawali</u>
Student's signature:	<u></u>
Student's signature:	<u>S.Patel</u>

COMP 3020 Social Web Analytics Assignment

Muhammed Mafawalla, Manan Bhatia, Rosmit Gyawali and Shekhar Patel

2023-10-20

Declaration

Student Name	Student Number	Contribution
Muhammed Mafawalla	22024540	25%
Manan Bhatia	22035587	25%
Rosmit Gyawali	22069030	25%
Shekhar Patel	22034335	25%

By including this statement, we the authors of this work, verify that:

- We hold a copy of this assignment that we can produce if the original is lost or damaged.
- We hereby certify that no part of this assignment/product has been copied from any other student's work or from any other source except where due acknowledgement is made in the assignment.
- No part of this assignment/product has been written/produced for us by another person except where such collaboration has been authorised by the subject lecturer/tutor concerned.
- We are aware that this work may be reproduced and submitted to plagiarism detection software programs to detect possible plagiarism (which may retain a copy on its database for future plagiarism checking).
- We hereby certify that we have read and understood what the School of Computing and Mathematics defines as minor and substantial breaches of misconduct as outlined in the learning guide for this unit.

Library Import

Topic Selection

We have selected the company **Apple Inc** as our topic for this assignment.

1. Question 1

Library Import

1.1 Identify relevant subreddits on Reddit, related to your chosen topic. Select three subreddits with the highest number of subscribers for the weekly period and get their URLs.

We will first gather subreddits related to our chosen topic and store it in a dataframe.

signature:

Note: An examiner or lecturer / tutor has the right to not mark this assignment if the above declaration has not been signed.

```
subreddits <- find_subreddits("Apple")
```

```
## parsing URLs on page 1...
```

```
## parsing URLs on page 2...
```

```
## Warning in file(con, "r"): cannot open URL
```

```
## 'https://www.reddit.com/subreddits/search.json?limit=100&q=Apple&after=t5_3i8bs':
```

```
## HTTP status was '429 Unknown Error'
```

Let us view the dimensions of the dataframe.

```
dim(subreddits)
```

```
## [1] 101 7
```

We have 163 subreddits and 7 features describing them.

Let us view the subreddit name and number of subscribers for the first five subreddits

```
head(subreddits[,c('subreddit', 'subscribers')])
```

```
##      subreddit subscribers
## 1          <NA>         NA
## 2qh1f      apple    4423274
## 2wav7 AppleWatch    788641
## 33hhf AppleMusic    159369
## 2qh2b      iphone    3894444
## 2s3df    appletv    478491
```

We will need to sort the subreddits by the number of subscribers in descending order. We will achieve this using the following code.

```
subredditsSorted <- subreddits[order(-subreddits$subscribers), ]
head(subreddits[,c('subreddit', 'subscribers')])
```

```
##      subreddit subscribers
## 1          <NA>         NA
## 2qh1f      apple    4423274
## 2wav7 AppleWatch    788641
## 33hhf AppleMusic    159369
## 2qh2b      iphone    3894444
## 2s3df    appletv    478491
```

Unfortunately, as shown above, the subreddits are irrelevant to our chosen topic of Apple Inc. Thus, we will design an algorithm to sort through the dataframe and retrieve subreddits that relate our chosen topic.

Let us create a list of keywords which we can use to filter our dataframe. These keywords will relate to the title of the subreddit.

We will select keywords that relate to Apple Inc physical products and software.

```
titleKeywords <- c("iphone", "ipad", "mac", "macbook", "appletv", "homepod", "watch", "applewatch", "wa
```

Next, we will retrieve the subreddits that contain the keywords above in the subreddit name.

```
# Filter the dataframe
subredditsSorted <- subredditsSorted[tolower(subredditsSorted$subreddit) %in% titleKeywords,]
# Display the top 3 subreddits sorted by amount of subscribers
head(subredditsSorted[,c('subreddit', 'subscribers')], 3)
```

```
##           subreddit subscribers
## 2qh2b      iphone    3894444
## 2qh4u       mac     2821090
## 2wav7 AppleWatch    788641
```

Thus, we will select the above three subreddits for Apple Inc:

- iPhone: <https://www.reddit.com/r/iPhone/>
- Mac: <https://www.reddit.com/r/mac/>
- AppleWatch: <https://www.reddit.com/r/AppleWatch/>

1.2 Retrieve the text (posts) from each of these subreddits.

Firstly, using the Reddit API, we will retrieve the subreddits using the `find_thread_urls()` function. A data set can be created with all of the attributes of the thread. Let's find top links for threads and comments for each subreddit posted in the last week.

Please note we have downloaded the threads and exported as a csv file for consistent analysis.

```
#iPhoneThreads <- find_thread_urls(subreddit = 'iphone', sort_by = 'top', period = 'week')
#watchThreads <- find_thread_urls(subreddit = 'AppleWatch', sort_by = 'top', period = 'week')
#macThreads <- find_thread_urls(subreddit = 'mac', sort_by = 'top', period = 'week')

iPhoneThreads <- read.csv("Data/iPhoneThreads.csv")
watchThreads <- read.csv("Data/watchThreads.csv")
macThreads <- read.csv("Data/macThreads.csv")
```

We can examine the structure and dimensions for each of the subreddit data.

```
class(iPhoneThreads)
```

```
## [1] "data.frame"
```

```
class(macThreads)
```

```
## [1] "data.frame"
```

```
class(watchThreads)
```

```
## [1] "data.frame"
```

Dimensions

```
dim(iPhoneThreads)
```

```
## [1] 1001    7
```

```
dim(macThreads)
```

```
## [1] 649    7
```

```
dim(watchThreads)
```

```
## [1] 747    7
```

The data has been returned as a data frame.

```
str(iPhoneThreads)
```

```
## 'data.frame':    1001 obs. of  7 variables:
## $ date_utc : chr   NA "2023-10-04" "2023-10-04" "2023-10-04" ...
## $ timestamp: int   NA 1696405290 1696404491 1696404301 1696402765 1696402156 1696401289 1696400793 1
## $ title    : chr   NA "Low call volume on Bluetooth earbuds, music youtube etc plays fine" "Charger c
## $ text     : chr   NA "Using a pair of generic Bluetooth earbuds (Redmi Buds), everything works quit
## $ subreddit: chr   NA "iphone" "iphone" "iphone" ...
## $ comments : int   NA 0 3 5 2 3 1 5 2 1 ...
## $ url      : chr   NA "https://www.reddit.com/r/iphone/comments/16zgxxq/low_call_volume_on_bluetooth
```

As we are only interested in the text data, we will extract the column `text` from our dataframe.

```
iPhone <- iPhoneThreads$text
watch <- watchThreads$text
mac <- macThreads$text
```

```
class(iPhone)
```

```
## [1] "character"
```

For each of our subreddits, we now have a character array representing the text data for each post.

1.2.1 Data Preprocessing

We will now preprocess our data to ensure we can conduct analysis on it. Using the `control = list()`, we will specify text cleaning functions. For example in the English language certain words that hold no significance such as “the”, “a”, “and” are called stopwords. Thus, we will remove them. Additionally we will use a stemming algorithm to reduce words to their base form. Words such as “retrieval”, “retrieved” will be reduced to the root word “retrieve”. Since our reddit text data may contain unwanted characters, we will apply a variety of functions such `removeNumber`, `removePunctuation` etc.

Since we will complete the data cleaning and processing for each subreddit, we will create a function in order to not repeat code.

```

textCleaning <- function(array) {
  corpus <- Corpus(VectorSource(array))
  corpus <- tm_map(corpus, function(x) iconv(x, to='UTF8', sub='byte'))
  tm <- tm_map(corpus, function(x) iconv(x, to='ASCII', sub=' ')) # remove special characters
  tm <- tm_map(tm, removeNumbers) # remove numbers
  tm <- tm_map(tm, removePunctuation) # remove punctuation
  tm <- tm_map(tm, stripWhitespace) # remove whitespace
  tm <- tm_map(tm, tolower) # convert all to lowercase
  tm <- tm_map(tm, removeWords, stopwords()) # remove stopwords
  tm <- tm_map(tm, stemDocument) # convert all words to their stems
  return(tm)
}

```

We now easily apply this custom function over all our subreddits.

1.2.2 Creating a Document Term Matrix (DTM)

We will first create the document term matrix where each record represents a document(in our case a reddit post) and each column represents a word. Moreover we will remove any empty reddit posts.

We will create a function for this transformation as well.

```

documentTermMatrix <- function(termMatrix, removalEmpties=TRUE) {
  dtm <- DocumentTermMatrix(termMatrix)
  if (removalEmpties) {
    empties <- which(rowSums(abs(as.matrix(dtm))) == 0)
    return(dtm[-empties,])
  }
  return(dtm)
}

```

Let us create the document term matrices for each our subreddits.

```

iPhone.dtm <- documentTermMatrix(iPhone.tm)
watch.dtm <- documentTermMatrix(watch.tm)
mac.dtm <- documentTermMatrix(mac.tm)

```

1.2.3 Creating a Term Document Matrix (TDM)

Next we will transpose our document term matrix into a term document matrix. Now, each row will represent a term and each column will represent a reddit post. We will also create a function for this processing.

```

termDocumentMatrix <- function(termMatrix, removalEmpties = TRUE) {
  tdm <- TermDocumentMatrix(termMatrix)
  if (removalEmpties) {
    empties <- which(colSums(as.matrix(tdm)) == 0)
    return(as.matrix(tdm[, -empties]))
  }
  return(tdm)
}

```

We will create the term document matrices for each our subreddits.

```

iPhone.tdm <- termDocumentMatrix(iPhone.tm)
watch.tdm <- termDocumentMatrix(watch.tm)
mac.tdm <- termDocumentMatrix(mac.tm)

```

1.3 Display the subreddit name and create a word cloud for the posts within each subreddit, resulting in a total of three-word clouds.

We will create the word cloud using term document matrix. Similar to the above, we will create a function to remove words that have “NA” as a count. We will also suppress the warnings and adjust specific arguments of the `wordcloud()` function such as word rotation, colors etc.

```

wordCloud <- function(tdm) {
  freqs = rowSums(tdm)
  freqs = freqs[!is.na(freqs)] # removal of words which have no frequency
  suppressWarnings(# supression of warnings
    wordcloud(names(freqs),
              freqs,
              random.order = FALSE, # plot words in decscending frequency
              rot.per = 0.35, # proportion words with 90 degree rotation
              colors = brewer.pal(8, "Dark2"), # coloring of words from least to most frequent
              max.words = 100, # plot 100 words
            )
  )
}

```

We will use the term frequencies instead of the weighted term frequencies(TF-IDF). As words which have a low frequency in documents will have a higher score, which is the opposite our objective.

1.3.1 iPhone

```
wordCloud(iPhone.tdm)
```




The word cloud primarily revolves around discussions about the iPhone and its features, hardware, software, and issues. It highlights the iPhone's significance, including its camera, battery, apps, iOS operating system, and recommendations for cases and phone models. This subreddit discussions emphasise the iPhone's value to both users and curious individuals.

1.3.2 Mac

```
wordCloud(mac.tdm)
```



```
reddit.text <- c(iPhone, mac, watch)
```

We will then apply our custom text cleaning function to remove stop words, strip whitespace etc on our combined subreddit data.

Using our custom function which removes empty entries we will create the term document matrix and document term matrix. We will add the argument `removalEmpties = FALSE` due to the `weightTfidf` requirement. We will handle this removal for empty data separately.

```
reddit.tdm <- termDocumentMatrix(reddit.corpus, removalEmpties = FALSE)
reddit.dtm <- documentTermMatrix(reddit.corpus, removalEmpties = FALSE)
```

We will now create a new column: “subreddit_id” in our document term matrix to store the values: 1,2,3 with each representing a particular subreddit: * iPhone: 1 * Mac: 2 * Watch: 3

We will extract the number of documents in each subreddit and create vector of values 1, 2 and 3.

```
subredditNumbers <- c(rep(1, dim(iPhone.dtm)[1]), rep(2, dim(mac.dtm)[1]), rep(3, dim(watch.dtm)[1]))
table(subredditNumbers)
```

```
## subredditNumbers
##    1    2    3
## 895 567 647
```

Perfect, the vector matches the number of documents in each subreddit’s term document matrix(after text cleaning). We will add this new column to the `reddit.matrix` created below.

2.1.2 TF-IDF Matrix

Next we will apply the TF-IDF weights and convert it into a document term matrix

We will remove any empty subreddit data for both the term document matrix and document term matrix

```
empties <- which(rowSums(abs(reddit.matrix)) == 0)
reddit.matrix <- reddit.matrix[-empties,]

emptiesTDM <- which(colSums(as.matrix(reddit.tdm)) == 0)
reddit.tdm <- reddit.tdm[,-emptiesTDM]
```

We will append the new subreddit number column to our weighted document term matrix

```
reddit.matrix <- cbind(reddit.matrix, subredditNumbers)
```

2.2 Next, apply K-means clustering to cluster the combined text

2.2.1 Euclidean Distance

K-means is a non-supervised machine learning algorithm used to cluster data. It is a centroid based clustering algorithm which partitions the data points into “k” amount of clusters.

We wish to compute the within sum of squares score(SSW) for each cluster between 1 and 15. The SSW scores measures the distance between each point and the clusters center.

We will first create an empty vector to store the SSW scores

```
SSW <- c()
```

We will then iterate over values of k from 1 to 15 generating the sum of squares within(SSW) to select the best number of clusters. We would like to minimise the SSW score.

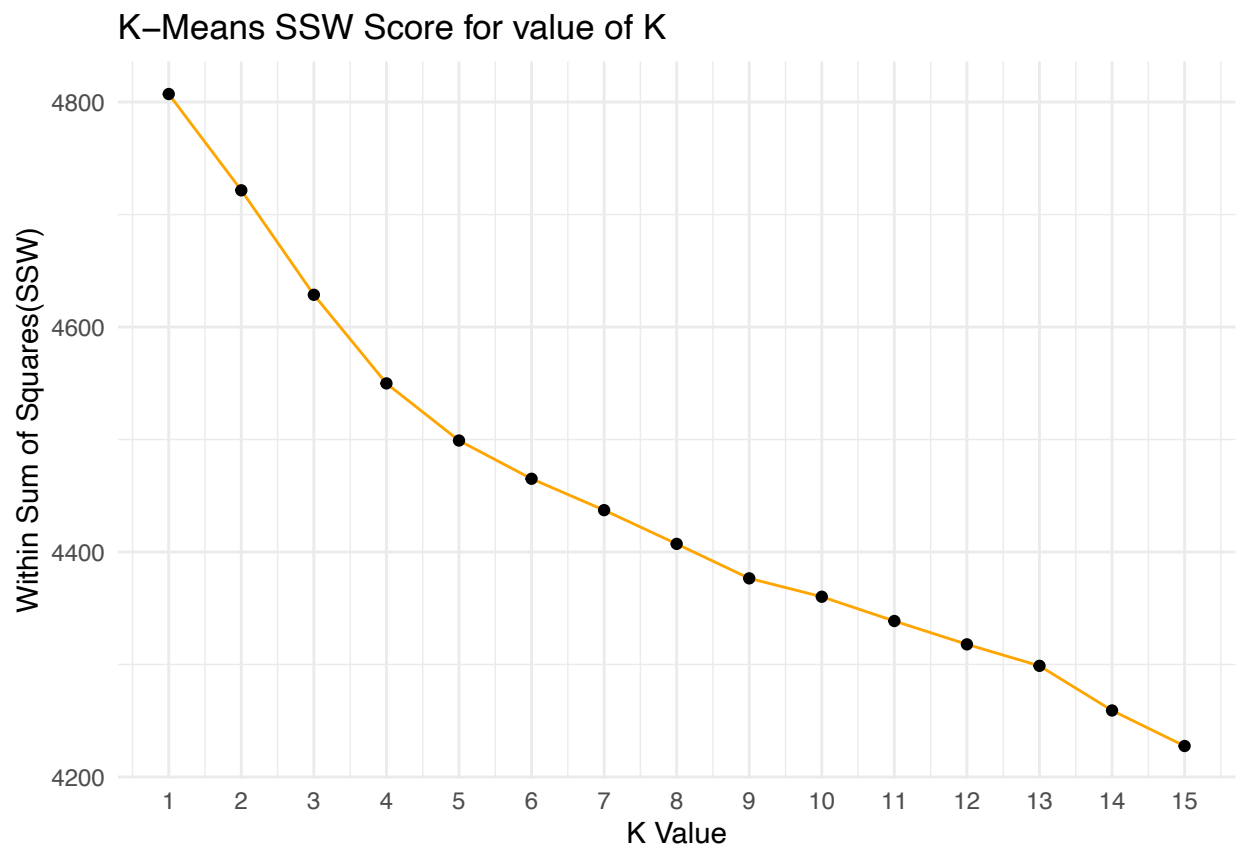
Since our matrix includes the subreddit number, we will select all columns except the last one using the following code: `reddit.matrix[, -ncol(reddit.matrix)]`

```
reddit.matrix.new <- reddit.matrix[, -ncol(reddit.matrix)]
```

```
for(k in 1:15){  
  K = kmeans(reddit.matrix.new, centers=k, nstart=3) # Iterate over values of k between 1 and 15  
  SSW[k] <- K$tot.withinss # Append the SSW score at k value to the vector  
}
```

We will then plot the SSW scores using ggplot.

```
ggplot(mapping = aes(x=1:15, y=SSW)) +  
  geom_line(color="orange") + # Line color to orange  
  geom_point() + # Add point for each value  
  theme_minimal() +  
  scale_x_continuous(breaks = 1:15, labels = 1:15) +  
  labs(title = "K-Means SSW Score for value of K", x="K Value", y="Within Sum of Squares(SSW)")
```



From the above plot, we can see there is an elbow at 5 clusters. However the SSW continues to drop for higher number of clusters. Given the large amount of clusters we can investigate using a different distance metric.

K-Means uses euclidean distance by default, which may not be appropriate for document vectors. We can investigate k-means with the cosine similarity function using multi-dimensional scaling.

2.2.2 Cosine Similarity with MDS

We will normalise the reddit matrix of weighted TF-IDF values to their unit length.

```
norm.reddit.matrix <- diag(1/sqrt(rowSums(reddit.matrix.new^2))) %*% reddit.matrix.new
```

We will then create the distance matrix using euclidean distance.

```
D <- dist(norm.reddit.matrix, method = "euclidean")^2/2
```

We will then perform multidimensional scaling using 100 dimensions

```
mds.reddit.matrix <- cmdscale(D, k=100)
```

The Reddit posts are now projected into a 100 dimensional euclidean space. We can visualise the SSW score for values of k using a line chart.

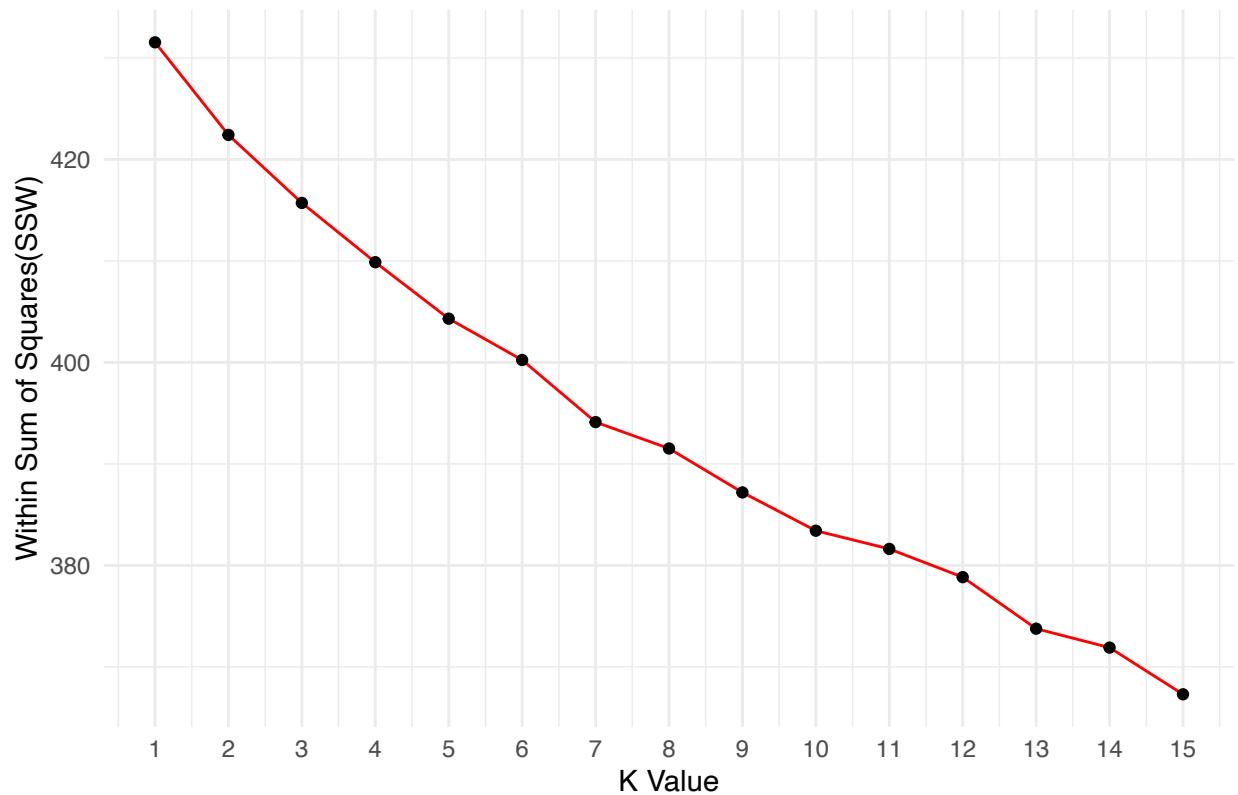
```
SSWMDS <- c() # Create the empty SSW scores vector
for (k in 1:15) { # Iterate over values of k between 1 and 15
  K = kmeans(mds.reddit.matrix, centers=k, nstart=3) # Apply the k-means clustering for value of k
  SSWMDS[k] <- K$tot.withinss # Append the SSW score at k value to the vector
}
```

```
## Warning: did not converge in 10 iterations
```

```
## Warning: did not converge in 10 iterations
```

```
ggplot(mapping = aes(x=1:15, y=SSWMDS)) +
  geom_line(color="red") + # Line color to red
  geom_point() + # Add point for each value
  theme_minimal() +
  scale_x_continuous(breaks = 1:15, labels = 1:15) +
  labs(title = "K-Means(Cosine Similarity) SSW Score for value of K", x="K Value", y="Within Sum of Squares")
```

K-Means(Cosine Similarity) SSW Score for value of K



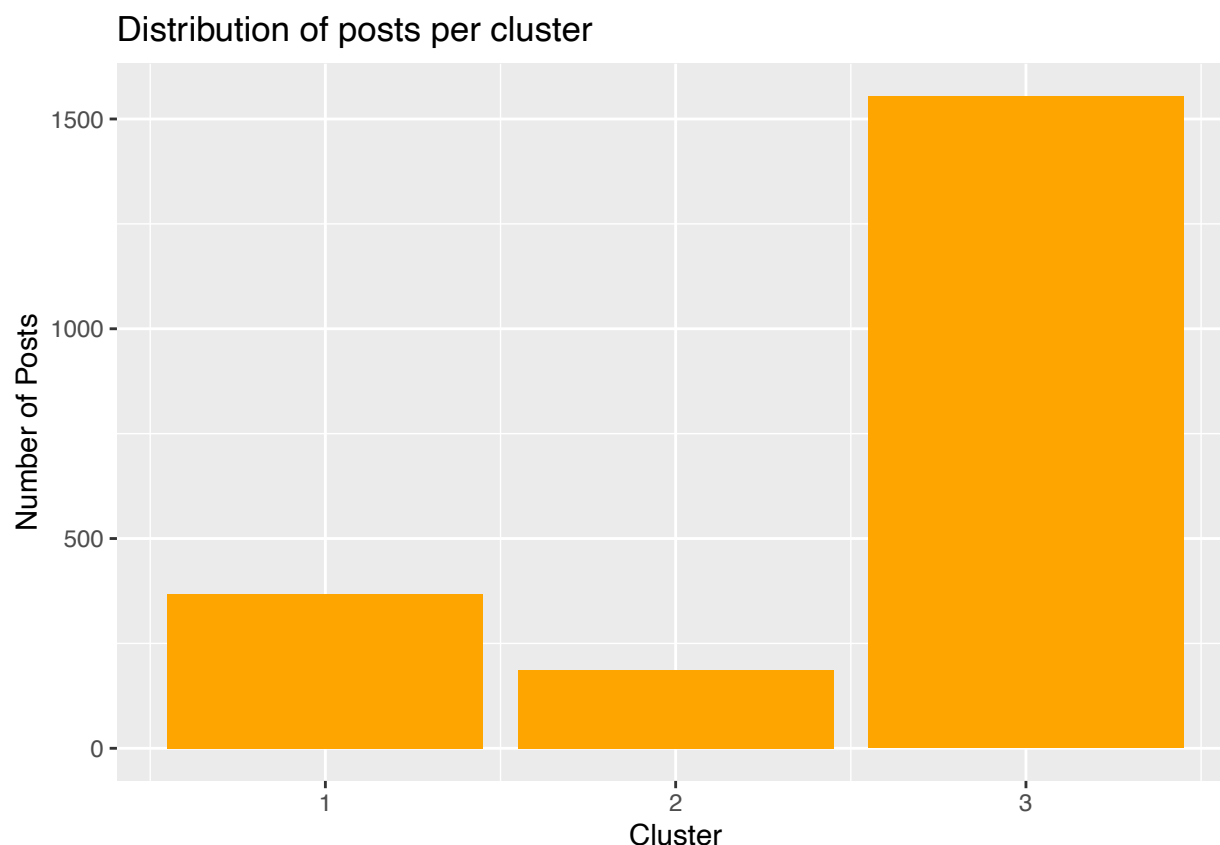
Once again, k-means was not effective in fully clustering our subreddits. Ideally we would expect a significant drop in SSW score after 3 clusters, as we have 3 different subreddits within our main topic. However, since both kmeans with and without cosine similarity retrieved similar results, we can conclude our subreddits share similarities between each other.

We will proceed with k-means with cosine distance and set the number of clusters to 3. Moreover, to reduce the impact of random initialisation we will set the `nstart` argument to 10.

```
K <- kmeans(mds.reddit.matrix, centers = 3, nstart = 10)
```

Let us view the distribution of points for each of our clusters.

```
ggplot(as.data.frame(table(K$cluster)), mapping=aes(x = 1:3, y = as.numeric(table(K$cluster)))) +
  geom_bar(stat = "identity", fill = "orange") +
  labs(title = "Distribution of posts per cluster", x = "Cluster", y = "Number of Posts")
```



Cluster two contains the most posts, followed by cluster three then lastly cluster one.

Topics discussed in iPhone, Mac and Apple Watch have a level of closeness. We can visualise these common topics by undergoing dimensionality reduction using multidimensional scaling on our dataset and plotting the results. We have selected multidimensional scaling over principal component analysis(PCA) due to the poor results obtained with PCA.

2.3 Visualise the results in two-dimensional vector space. Ensure your visualization displays both the clusters and the original labels of the documents (where each text is considered a document).

2.3.1 Dimensionality Reduction

In order to visualise the data, we need to reduce it to two dimensions. We will employ multidimensional scaling(MDS) with binary and cosine distance to achieve this.

Multidimensional scaling(MDS) uses pairwise relationship to represent the similarity of data. MDS can use a variety of distance measures such as cosine, binary and even euclidean distance.

2.3.3 Binary Distance using unweighted posts

Next, we will compute the binary distance using the unweighted matrix. The binary distance will examine if elements of a vector are present or absent, while ignoring the weight of the element.


```
M <- as.matrix(reddit.tdm) # Convert term document to matrix
D <- dist(t(M), method = "binary") # Compute the binary distance
mdsM <- cmdscale(D, k=2) # Scale to 2 dimensions
```

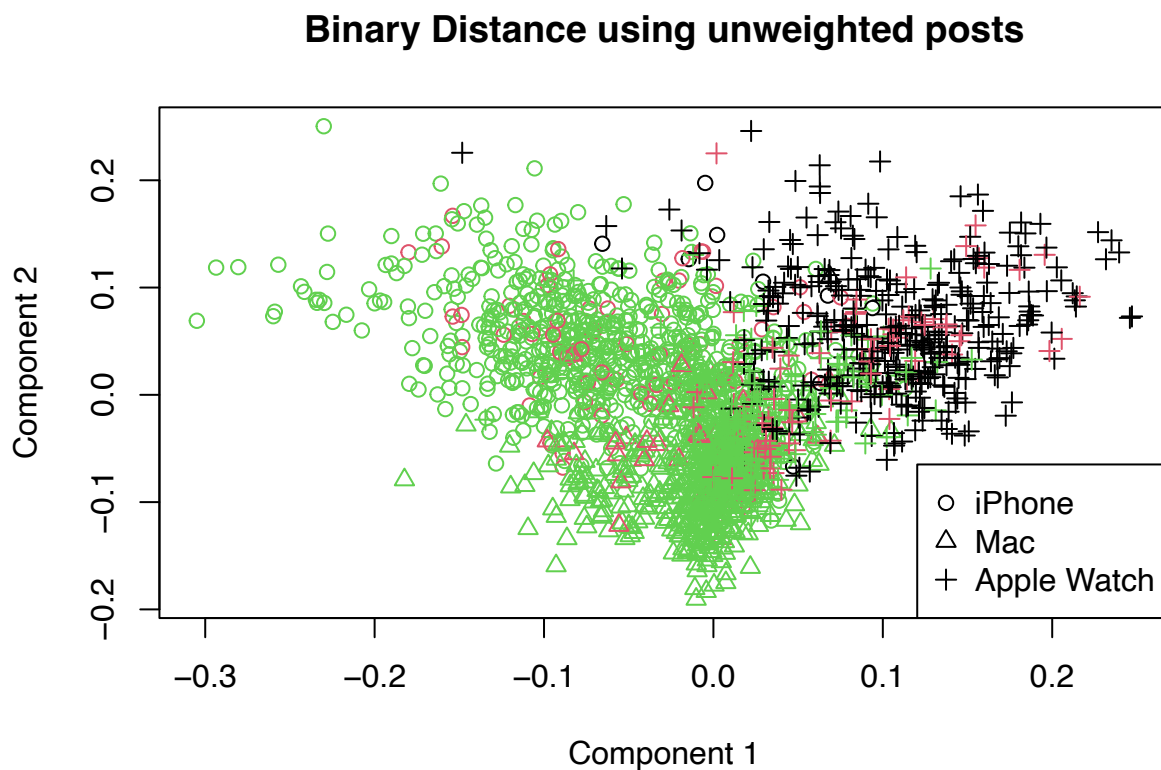
We will plot the data using the first two components. The `col` argument will be the K-Means clusters and the point `pch` will be set to the original labels: 1,2 and 3, stored as the last column in the document term matrix `reddit.matrix[, ncol(reddit.matrix)]`.

Let us extract the original labels of the subreddits.

```
originalLabels <- reddit.matrix[, ncol(reddit.matrix)]
```

We can then plot our binary distance with unweighted posts

```
plot(mdsM[,1], mdsM[,2], col=K$cluster, pch = as.numeric(originalLabels), main="Binary Distance using unweighted posts",
legend("bottomright", legend=c("iPhone", "Mac", "Apple Watch"), pch = c(1,2,3))
```



From the above plot, we can generally cluster each of the subreddits although there is some overlap. Posts of Mac tend to be concentrated on certain topics and clump together. Moreover, Apple Watch and iPhone seem to diverge from each other indicating their unique characteristics, one being a phone and another a watch

Regarding the overlap seen at the center(0,0), we can infer this is due to common topics between the subreddits such as Apple, screen, battery life etc. The overlap between Apple Watch and iPhone could contain topics discussing pairing and communication between these devices.

Furthermore, kmeans did not cluster the points effectively as there is mislabeling of data points. iPhone and Mac data points have been majorly grouped as one cluster while some Apple Watch points have been mislabelled.

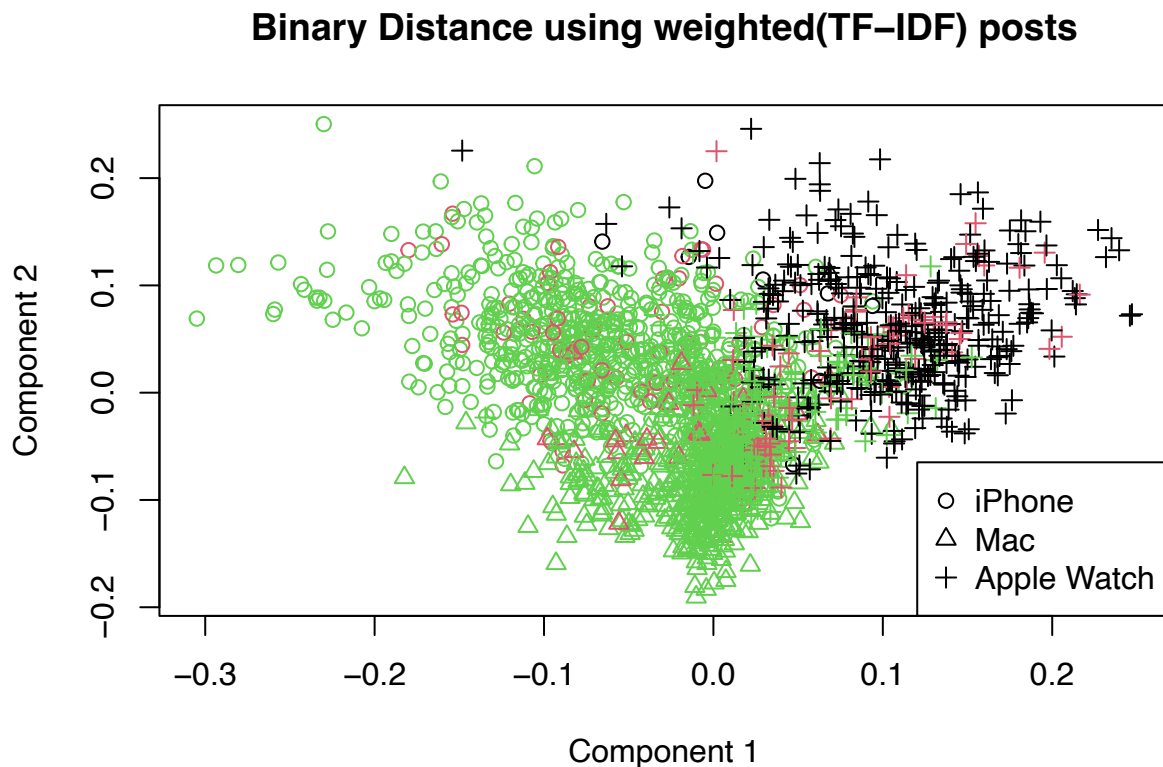
2.3.4 Binary Distance using weighted(TF-IDF) posts

We will now compute the binary distance using the weighted(TF-IDF) posts.

```
D <- dist(reddit.matrix.new, method = "binary")
mdsT <- cmdscale(D, k=2) # scale to two dimensions
```

We can then visualise the results.

```
plot(mdsT[,1], mdsT[,2], col=K$cluster, pch = as.numeric(originalLabels), main="Binary Distance using w
legend("bottomright", legend=c("iPhone", "Mac", "Apple Watch"), pch = c(1,2,3))
```



The plots of weighted and unweighted binary distances appear similar and thus our analysis above would apply.

2.3.5 Cosine distance with unweighted posts

We will convert the term document matrix to a matrix data type. Next we will center and scale the matrix through standardisation.

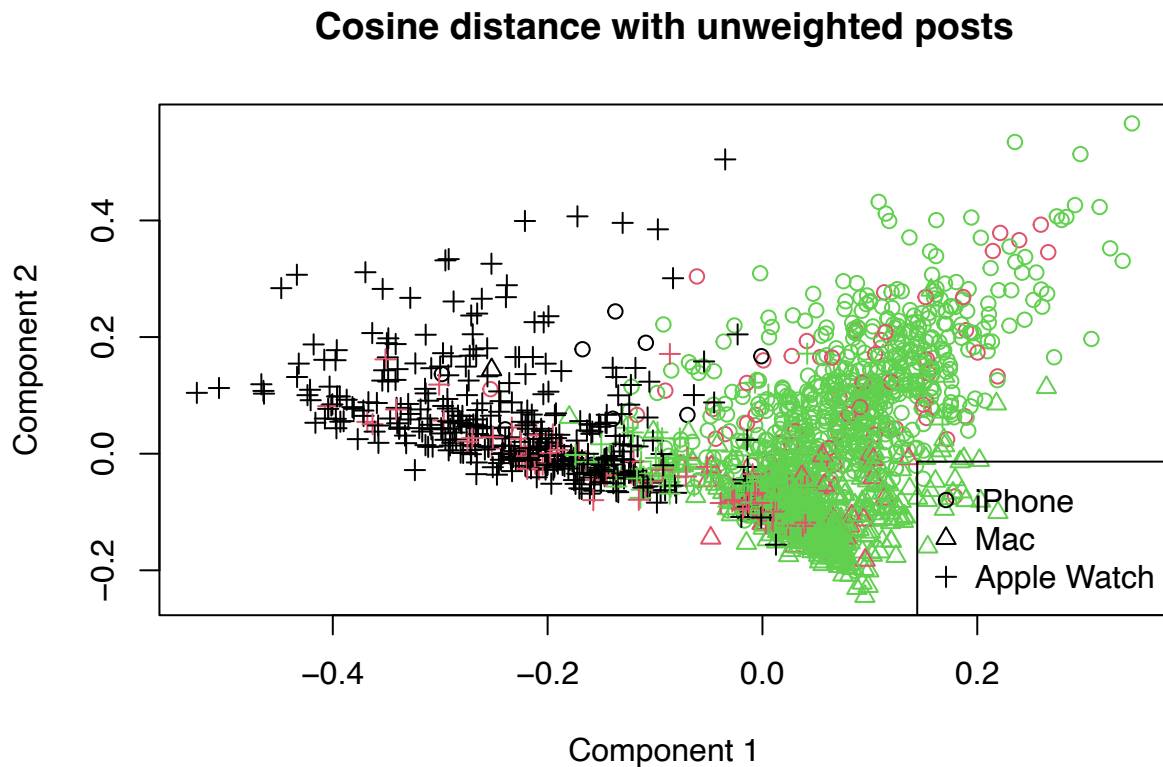
```
M <- as.matrix(reddit.tdm)
CM <- M %*% diag(1/sqrt(colSums(M^2)))
```

We will then calculate the pairwise euclidean distances and lastly perform multidimensional scaling on the distances

```
D <- dist(t(CM), method = "euclidean")^2/2
mdsM <- cmdscale(D, k=2)
```

We can now plot the Cosine distance with unweighted posts.

```
plot(mdsM[,1], mdsM[,2], col=K$cluster, pch = as.numeric(originalLabels), main="Cosine distance with unweighted posts",
legend("bottomright", legend=c("iPhone", "Mac", "Apple Watch"), pch = c(1,2,3))
```



Using the unweighted version of cosine distance produces a difference in the spread of Apple Watch. The data points of Apple Watch are widely dispersed along the first component along the x axis. Moreover, similar to the binary distance, Mac forms a tight cluster around (0,0) however it scatters along with iPhone. Furthermore, iPhone and Apple Watch move in opposite directions similar to the binary distance.

Similarly to the binary distance, k-means did a poor job clustering the posts due to points having different colours.

2.3.1 Clusters Terms

We can examine the words belonging to each cluster. We will create a function to automate this retrieval.

```
retrieveTermsAtCluster <- function(clusterNumber, numberTerms, K, reddit.matrix) {
  postsID = which(K$cluster == clusterNumber)
  clusterPosts = reddit.matrix[postsID,]
  clusterTermWeight = colMeans(clusterPosts)
  sort(clusterTermWeight, decreasing = TRUE)[1:numberTerms]
}
```

We can now retrieve the words at cluster 1.

```
retrieveTermsAtCluster(clusterNumber = 1, numberTerms = 10, K, reddit.matrix.new)
```

```
##      watch      appl      face      seri      show      app      workout
## 0.15083568 0.06697299 0.03389698 0.03380481 0.03190496 0.02723192 0.02682090
##      ultra      phone      get
## 0.02658233 0.02558142 0.02362491
```

Cluster one appears to be common topics between the devices such as batteries, charging, update etc.

Let us view cluster 2:

```
retrieveTermsAtCluster(clusterNumber = 2, numberTerms = 10, K, reddit.matrix.new)
```

```
##      batteri      charg      life      day      charger      drain      updat
## 0.16245378 0.14330901 0.05174093 0.04781788 0.04538554 0.03712382 0.03460561
##      use      hour      watch
## 0.03411489 0.03294121 0.03169793
```

Cluster two appears to include terms associated with iPhone, such as phone, iPhone, the iPhone Pro etc.

Let us view cluster 3:

```
retrieveTermsAtCluster(clusterNumber = 3, numberTerms = 10, K, reddit.matrix.new)
```

```
##      iphon      pro      use      phone      can      just      anyone
## 0.03288182 0.02823059 0.02420639 0.02215772 0.02108755 0.02100266 0.02094994
##      app      work      screen
## 0.02019241 0.02015537 0.01972969
```

Cluster three seems to be centered around Apple Watch with terms such as watch, watch series, workouts etc.

2.4 Comment on your results. Assess the performance of K-means clustering in terms of correctly identifying the clusters. Did it effectively capture the underlying patterns?

Given the above comments on the kmeans visualisations and above cluster terms. K-means did not perform well but rather satisfactory at clustering the terms. As seen above, each of the 3 clusters share topics such as phone and watch which demonstrates the poor clustering of the subreddit data.

Ultimately, k-means performed satisfactory in the clustering of terms, however it failed to correctly identify the clusters, as mentioned above and in the previous sections. Moreover, it failed to capture the underlying patterns of the data.

Question 3

3. Examine whether there exists a linear relationship between the number of comments in a subreddit and the date. Note: You might need to format the date in your dataset appropriately. Visualize your findings by plotting the data along with the regression line.

We will select the iPhone subreddit to answer this question and question four.

3.1 Hypothesis test for the slope of the line

Apply the **hypothesis test for the slope of the line** as we are testing the **significance** of a single slope coefficient.

We use a **hypothesis test** to determine if the slope coefficient(b) in a linear regression model is significantly different from zero.

Firstly, let's state the hypothesis

3.1.1 Step 1

$H_0 : b = 0$ (there is no linear relationship, which means the slope coefficient (b) is equal to zero)

$H_1 : b \neq 0$ (there is a linear relationship, which means the slope coefficient (b) is not equal to zero)

3.1.2 Step 2

$\alpha = 0.05$. Reject if $p\text{-value} < 0.05$

3.1.3 Step 3

Convert the date to numeric format for regression

```
iPhoneThreads$dateNumeric <- as.numeric(as.Date(iPhoneThreads$date_utc))
```

Fit a linear regression model

```
linRegression <- lm(comments ~ dateNumeric, data=iPhoneThreads)
```

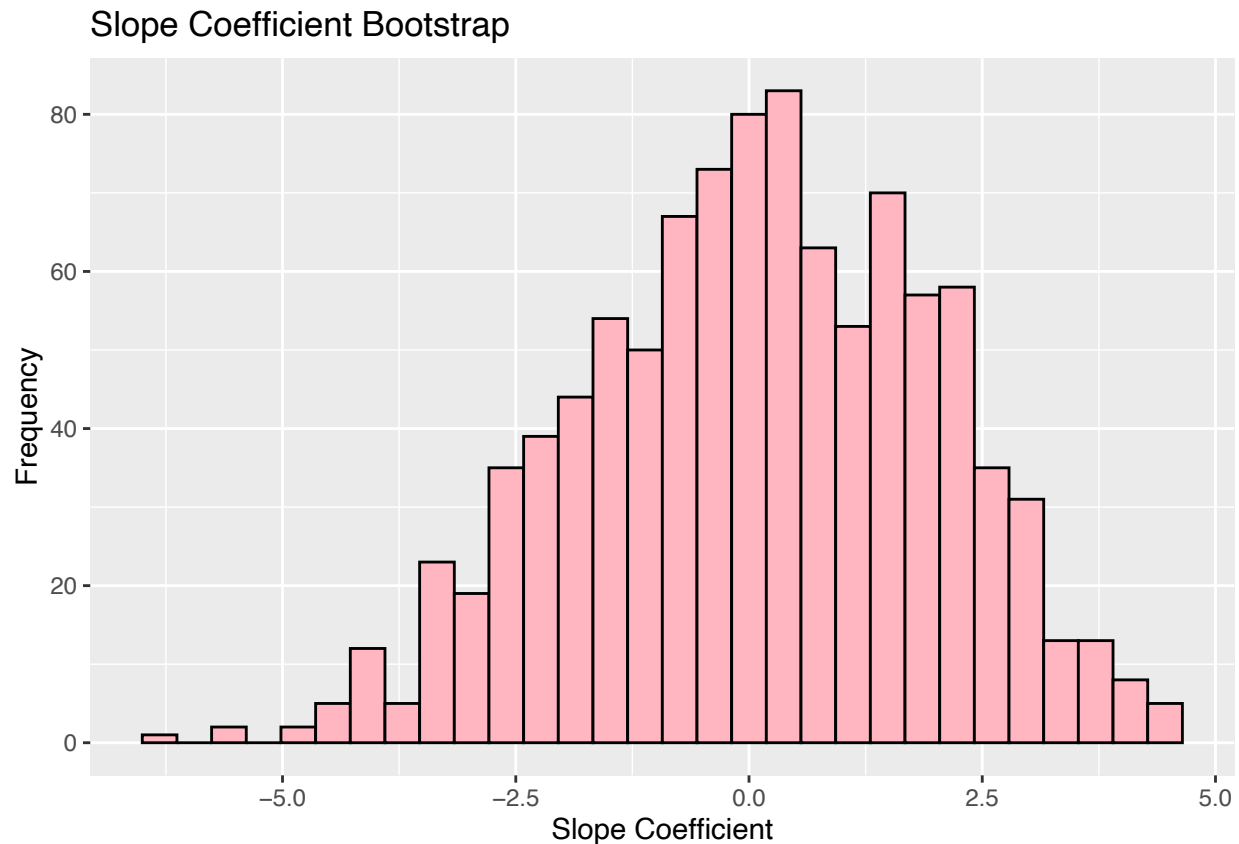
Create a bootstrap distribution of b where the null hypothesis is true ($b = 0$)

```
boot.dist <- replicate(1000, {  
  comments.perm <- sample(iPhoneThreads$comments) # shuffle one variable to force population b = 0  
  fit <- lm(comments.perm ~ iPhoneThreads$dateNumeric) # fit the straight line model  
  coef(fit)[2] # return the fitted b  
})
```

Examine the distribution of b , when the population $b = 0$

```
ggplot(as.data.frame(boot.dist), aes(x = boot.dist)) +
  geom_histogram(color="black", fill="lightpink") +
  labs(title="Slope Coefficient Bootstrap", x="Slope Coefficient", y="Frequency")
```

```
## 'stat_bin()' using 'bins = 30'. Pick better value with 'binwidth'.
```



We can now extract the slope from our linear regression model.

```
slope <- coef(linRegression)[2]
slope
```

```
## dateNumeric
## -5.864167
```

We can compute the probability of getting the slope b , if the population $b = 0$

```
pValue <- mean(boot.dist > (-slope)) + mean(boot.dist < slope)
pValue
```

```
## [1] 0.001
```

3.1.4 Step 4

Since $p\text{-value} < 0.05$, we can reject H_0 . There is evidence of an association between the number of comments and the date ($b \neq 0$).

3.2 Visualize your findings by plotting the data along with the regression line.

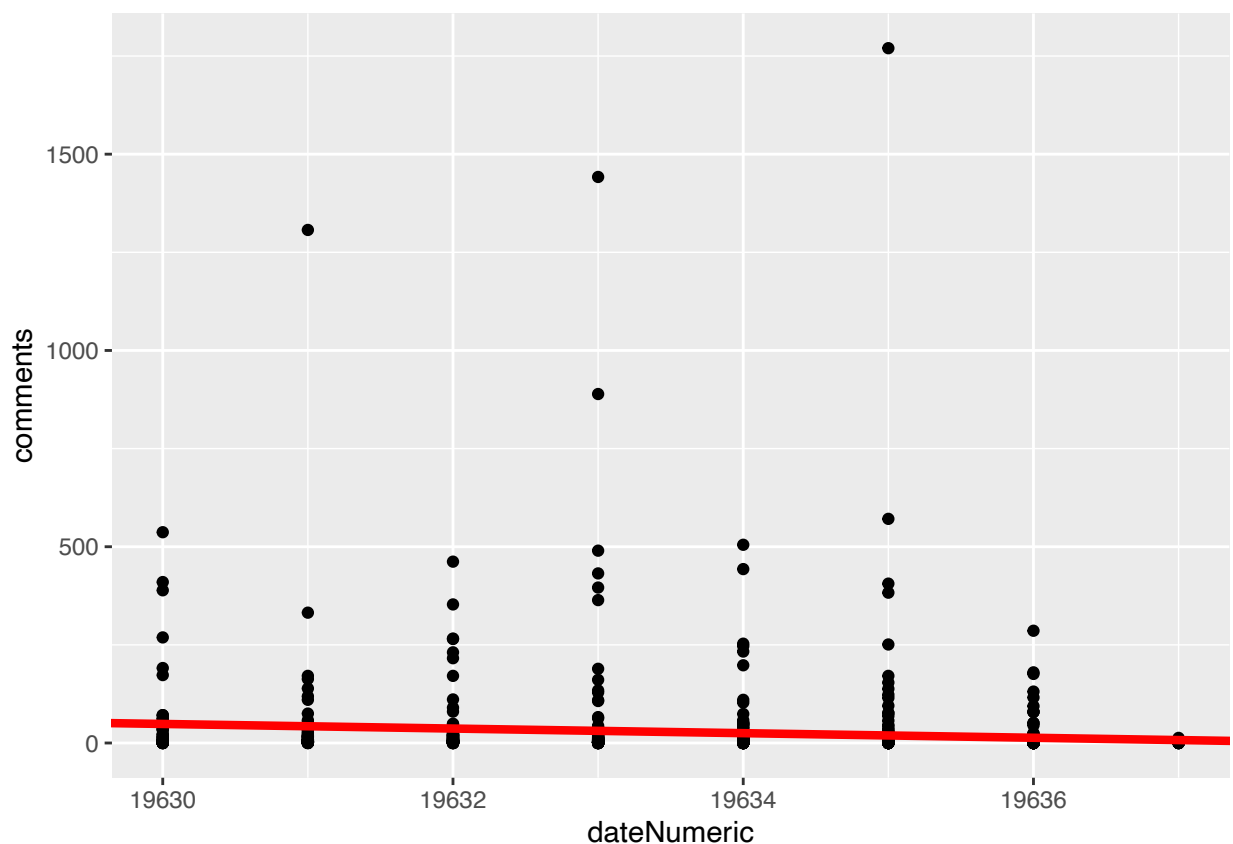
We will first extract the coefficients from our linear regression model.

```
intercept <- coefficients(linRegression)[1]
slope <- coefficients(linRegression)[2]
```

We will plot the data along with a regression line using the above coefficients.

```
ggplot(iPhoneThreads, aes(x = dateNumeric, y = comments)) +
  geom_point() + geom_abline(intercept = intercept, slope = slope, color="red",
                             linetype="solid", linewidth=1.5)
```

```
## Warning: Removed 1 rows containing missing values ('geom_point()').
```



From the above plot there is evidence the number of comments is negatively correlated with the date. As the date increases, the number of comments decrease. This is confirmed by the negative slope value.

Question 4

4. Perform a test to evaluate whether the number of threads is equally likely for each day

Let us view the days of our posts

```
unique(iPhoneThreads$date_utc)
```

```
## [1] NA "2023-10-04" "2023-10-03" "2023-10-05" "2023-10-06"  
## [6] "2023-10-02" "2023-10-01" "2023-09-30" "2023-10-07"
```

4.1 Data Visualisation

Convert the date to day of the week

```
iPhoneThreads$dayOfWeek <- weekdays(as.Date(iPhoneThreads$date_utc))
```

Count how many threads were created on each day

```
threadCountByDay <- table(iPhoneThreads$dayOfWeek)  
threadCountByDay
```

```
##  
##    Friday    Monday  Saturday    Sunday  Thursday  Tuesday Wednesday  
##      211       85       87       71      194      135      217
```

We can visualise the above thread distribution using a bar plot.

We will first convert the above table into a dataframe, then rename the columns.

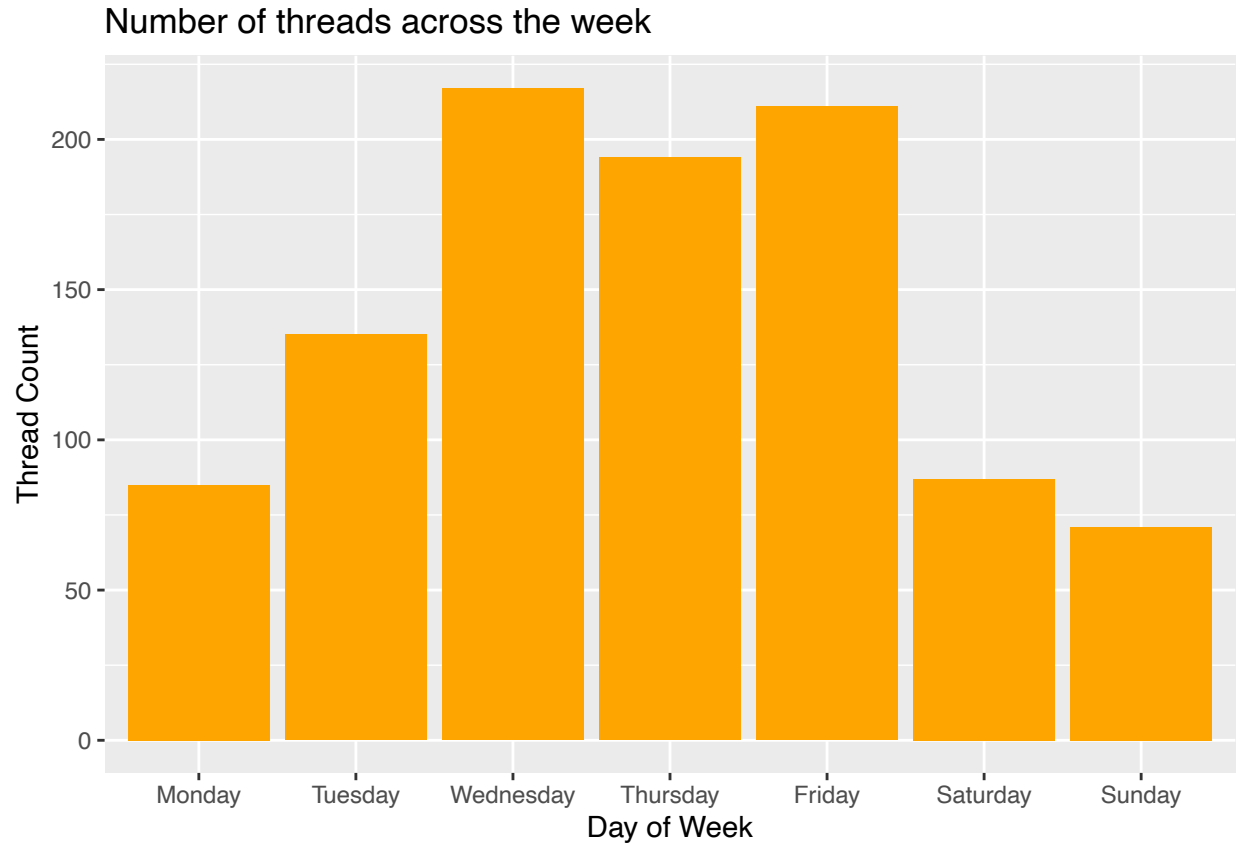
```
threadCountByDayDF <- as.data.frame(threadCountByDay)  
names(threadCountByDayDF)[1] <- "DayOfWeek"  
names(threadCountByDayDF)[2] <- "Count"
```

We will also adjust the factor levels of day of week and reorder respectively.

```
threadCountByDayDF$DayOfWeek = factor(threadCountByDayDF$DayOfWeek, levels= c("Monday",  
  "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"))
```

We will then plot the visualisation using a bar plot.

```
ggplot(threadCountByDayDF, mapping=aes(x=DayOfWeek, y=Count)) +  
  geom_bar(stat='identity', fill="orange") +  
  labs(title="Number of threads across the week", x="Day of Week", y="Thread Count")
```

From the above plot, there is evidence the number of threads posted across the weeks differs. However we can test this statement using a chi-squared goodness of fit test.

4.2 Chi Squared Goodness of Fit

We will use the iPhone subreddit for this question.

We need to apply the **chi squared goodness of fit test** for a uniform distribution as there is only one categorical variable in the population (**number of threads**). The test determines if sample data corresponds with the hypothesised distribution (Libretexts, 2023).

4.2.1 Step 1

H_0 : The number of threads is equally likely for each day ($p_{2023-09-30} = p_{2023-10-01} = p_{2023-10-02} = p_{2023-10-03} = p_{2023-10-04} = p_{2023-10-05} = p_{2023-10-06}$)

H_1 : The number of threads is not equally likely for each day

4.2.2 Step 2

$\alpha = 0.05$. Reject if p-value < 0.05

4.2.3 Step 3

```
chiSquaredTest <- chisq.test(threadCountByDay)
```

We will verify our expected counts

```
chiSquaredTest$expected
```

```
##      Friday      Monday  Saturday      Sunday  Thursday      Tuesday  Wednesday
##  142.8571  142.8571  142.8571  142.8571  142.8571  142.8571  142.8571
```

Since our expected counts is greater than 5. We can proceed and view the output of our chi-squared test.

```
chiSquaredTest
```

```
##
##  Chi-squared test for given probabilities
##
## data:  threadCountByDay
## X-squared = 171.14, df = 6, p-value < 2.2e-16
```

Since the p-value is less than 0.05, we reject H_0 . There is evidence the number of threads is not equally likely for each day.

Mastodon Analysis

Background:

What is Mastodon

Mastodon is a decentralized social network made up of independent servers organized around specific themes, topics, or interests. People can join servers, follow each other, engage in conversations, and otherwise do all sorts of things they'd expect to do on a social network like Twitter.

Mastodon has been around since March 2016 but it only really blew up in late 2022 when Elon Musk bought the Twitter platform.

How does Mastodon work?

The biggest difference between Mastodon and Twitter can be summed up in a single word: decentralization. Social media platforms like Twitter are centralized, meaning they're owned and operated by a single company. That company builds features, moderates content, tweaks algorithms, and handles all other tasks that come with running a social network.

Comparatively, while there are people at a non-profit company called "Mastodon gGmbH" that work on the social platform known as Mastodon, the platform itself isn't centralized within that company. That's because it's made up of thousands of independent servers. Think of a Mastodon server as a mini social network or a forum. Usually, a server is organized around a specific interest, topic, or industry. Once you've joined a server, you can follow, reply to, and engage with anyone on Mastodon, no matter what server they're on.

Mastodon is a network of servers, rather than a single social platform, so businesses can't go through a single person or platform to "do marketing" on Mastodon. Additionally, Mastodon's homepage states that the service will never serve ads or push some profiles above others. Since Mastodon is run by a non-profit, there's no profit incentive and no desire to work with advertisers.

In Mastodon, each server is essentially a self-contained community gathered around a single topic, whether that's locality, a shared interest, or even a specific career. Find the right server, and you can get unfiltered access to what really matters for your target demographic or a new market segment. (Hootsuite 2022)

Analysis:

```
#library import
```

```
library(rtoot)
```

```
## Warning: package 'rtoot' was built under R version 4.2.3
```

```
library(knitr)
```

```
## Warning: package 'knitr' was built under R version 4.2.3
```

```
library(igraph)
```

```
## Warning: package 'igraph' was built under R version 4.2.3
```

```
##
```

```
## Attaching package: 'igraph'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
##      decompose, spectrum
```

```
## The following object is masked from 'package:base':
```

```
##
```

```
##      union
```

Authentication setup:

```
#auth_setup()
```

5.i Using Mastodon, identify users who are relevant to your chosen topic.

To identify the users who are relevant to our topic, i.e, “apple”, we use the “search_accounts()” function. This function helps us to discover and retrieve the user accounts based on our keyword(topic), in our case, it is “apple”.

```
users = search_accounts("apple")  
dim(users) #gives the dimension of users df
```

```
## [1] 40 21
```

Now we have the ‘users’ dataframe obtained from the above ‘search_accounts()’ function, which has 40 rows and 21 columns. This users dataframe contains the details about our targeted users. To see what sort of details does this dataframe contains about our users, we can make use of names() function.

```
names(users)
```

```
## [1] "id"           "username"      "acct"          "display_name"  
## [5] "locked"       "bot"           "discoverable"  "group"  
## [9] "created_at"   "note"          "url"           "avatar"  
## [13] "avatar_static" "header"        "header_static" "followers_count"  
## [17] "following_count" "statuses_count" "last_status_at" "fields"  
## [21] "emojis"
```

We can see, it has the details like, username, display_name, id, followers_count, following_count and many more.

To further confirm the relevance of the accounts obtained from the search_accounts() function to our topic, we can check the notes or profile descriptions mentioned on our users' profiles.

```
o=order(users$followers_count, decreasing = TRUE) # sorts in decreasing order and prints the row ids.
acc_note=users$note[o[1:5]] #(bio)
acc_note
```

```
## [1] "<p><strong>Welcome</strong></p>\n<p>to the largest Apple community on Lemmy. This is the place v
## [2] "<p>In-depth Apple news, analysis & reviews since 1997</p>"
## [3] ""
## [4] "<p>Here you can talk about Apple's ecosystem, Apple's OSs (Operating Systems), Apple's apps, Ap
## [5] "<p>Todo sobre el universo Apple: macOS, iPod, iPhone, iPad, Apple Watch, Apple TV y Macs. Tambi
```

From the output above, it seems accounts are relevant to our topic.

5.ii Identify the top five most popular users, based on the highest number of followers.

```
top_accs_ids=users$id[o[1:5]] # save the id of top 5 users with the highest number of followers.
top_accs_username=users$username[o[1:5]] # save the username
top_accs_act_name=users$acct[o[1:5]] # save the accountname
```

We've stored key information for our top five users, including their username, user ID, and account name. Below, we'll present these details in a tabular format:

```
#dfs the top user data
top_users_data <- data.frame(
  ID = top_accs_ids,
  Username = top_accs_username,
  AccountName = top_accs_act_name
)

knitr::kable(top_users_data)
```

ID	Username	AccountName
110517322444957183	apple_enthusiast	apple_enthusiast@lemmy.world
109525175228746087	appleinsider	appleinsider
110528064055026327	apple	apple@lemmy.world
110499978865190837	apple	apple@lemmy.ml
109703819908045678	applesfera	applesfera

From the output above, we can say the following are our top five most popular users, based on the highest number of followers:

apple_enthusiast appleinsider apple apple applesfera

For our top third and fourth user, we can see both of them share the same username, which is possible on platforms like Mastodon. Unlike Twitter or Instagram, Mastodon allows identical usernames, but the uniqueness is maintained by distinct server domains. For instance, the third user in our list has the username “apple,” but we can see in table above that their account name is “apple@lemmy.world,” while the fourth user uses “apple@lemmy.ml.” The shared username is supplemented with different server domains, ensuring each account’s individuality. Mastodon’s approach might be less familiar due to its lower popularity. This concept is similar to email addressing.

5.iii) Download 50 followers and 50 friends (the users followed by your selected users) for each of these top five users.

Let’s first check for all of our top five users, if they’ve sufficient(50 in our case) followers:

```
# dfs with usernames and their follower counts
follower_counts <- users$followers_count[o[1:5]]
follower_table <- data.frame(Username = top_accs_username, Followers = follower_counts)

kable(follower_table, format = "markdown")
```

Username	Followers
apple_enthusiast	15382
appleinsider	6092
apple	4818
apple	4615
applesfera	4013

From the table above, we can see we’ve enough followers count that we need for our analysis for all of our top five users.

Let’s access the list of few followers for our top one user, i.e, apple_enthusiast:

```
apple_enthusiast_followers <- get_account_followers(top_accs_ids[1], limit = 10)
dim(apple_enthusiast_followers)
```

```
## [1] 0 0
```

Unfortunately, we’re not getting the list of followers for this user.

Let’s check for another user, applesfera,:

```
applesfera_followers <- get_account_followers(top_accs_ids[5], limit = 10)
dim(applesfera_followers)
```

```
## [1] 10 21
```

But for this user, we’re getting the list of followers.

So, what could be the issue?

After carefully investigating the issue, we've found that mastodon allows its users to adjust their privacy settings for followers, making them either public or private. If these lists are set to private, we are only able to see a count of followers, but not the actual list or details of followers.

That's what happened in the above scenario, apple_enthusiast has set its followers list to private because of which we can only access its followers count but the actual list of followers.

And it's same case with the followings as well.

To account for this Mastodon feature, we've to select our users who have enough followers and followings and also allows access to their followers and followings list.

Within our 'apple' topic, we observed that among the top ten users, only three have made their follower and followings lists public. Consequently, we could only access the follower and followings details for three users only from this group. By expanding our analysis to the top 15, we can include five users, comprising the initial three from the top 10 and an additional two from the top 15, who have publicly accessible follower and following lists.

Considering this condition, we formulate our code to get followers and friends data in a way that it iterates through the top15 users and collects the followers and followings of those users who allows us to access their list. We've introduced the break condition as well inside our loop which will break our loop once it has got the required(5 users in our case) users' followers and followings list. And it'll store user's followers and followings in their respective individual dataframe.

ids of top 15 users with the highest number of followers:

```
top_15accs_ids=users$id[o[1:15]]
```

collect followers and friends list:

```
# initialize empty lists
followers_df_list <- list()
followings_df_list <- list()

# initialize variables
num_users_collected <- 0
max_users <- 5

# iterate through the top 15 users
for (user_id in top_15accs_ids) {
  if (num_users_collected >= max_users) {
    break
  }

  followers_data <- get_account_followers(user_id, limit = 50)
  followings_data <- get_account_following(user_id, limit = 50)

  if (is.null(followers_data)
      || nrow(followers_data) == 0
      || is.null(followings_data)
      || nrow(followings_data) == 0) {
    # skip this user if data is empty
    next
  }

  user_name <- users$username[which(users$id == user_id)]
  cat("Collecting data for user:", user_name, "\n")
}
```

```

# store dfs in lists
followers_df_list[[user_name]] <- followers_data
followings_df_list[[user_name]] <- followings_data

# save dfs as individual variables
assign(paste0(user_name, "_followers_df"), followers_data)
assign(paste0(user_name, "_followings_df"), followings_data)

num_users_collected <- num_users_collected + 1
}

```

```

## Collecting data for user: appleinsider
## Collecting data for user: applesfera
## Collecting data for user: AppleFanboy
## Collecting data for user: Applelianos
## Collecting data for user: appleabear

```

Now, from the output above we can see that, we've successfully collected the list of followers and friends for our each five users from the list of top fifteen. And also we've avoided the users who've zero followers or followings.

And we've created the individual followers and friends dataframes for all of these five users which consist of their followers and friends list. Even though, we've applied the limit=50, to get only 50 list, for some users we've get more than 50 and for some less than 50. It's because some accounts are not discoverable hence less than the limit may be downloaded. On the other hand, when we prefer to download 50 friends automatic pagination is used. We may get more results than requested. And also for few of our users may not have the enough (50) followings or followers.

Let's visualise the count of followers and the followings that we've collected:

define the dataframes and usernames:

```

# Create a vector of usernames
usernames <- c("appleinsider", "applesfera", "AppleFanboy", "Applelianos", "appleabear")

# lists of followers and followings dfs
followers_df_list <- list(
  appleinsider_followers_df,
  applesfera_followers_df,
  AppleFanboy_followers_df,
  Applelianos_followers_df,
  appleabear_followers_df
)

following_df_list <- list(
  appleinsider_followings_df,
  applesfera_followings_df,
  AppleFanboy_followings_df,
  Applelianos_followings_df,
  appleabear_followings_df
)

```

plot the table with count:


```

follower_counts <- integer(length(usernames))
following_counts <- integer(length(usernames))

# row counts for followers and followings
for (i in seq_along(usernames)) {
  user <- usernames[i]
  follower_counts[i] <- nrow(followers_df_list[[i]])
  following_counts[i] <- nrow(following_df_list[[i]])
}

#plot table
user_counts_df <- data.frame(Usernames = usernames,
                             Followers_Counts = follower_counts,
                             Followings_Counts = following_counts)
kable(user_counts_df, format = "markdown")

```

Usernames	Followers_Counts	Followings_Counts
appleinsider	80	9
applesfera	80	5
AppleFanboy	80	75
Applelianos	80	17
appleabear	17	3

From the table above, we can see the following counts for most of our users is not enough, it's because these accounts don't have the enough followings itself. And for followers we can see we got 80 followers for 4 users even though we applied the limit = 50, it's probably because of the pagination.

5.iv Create a graph and visualize the relationships among these users.

In order to create a graph, we first need to create an edgelist of our followers and friends.

5.iv.i Data Pre-processing: creating edgelist

function to create an edgelist:

```

create_edgelist <- function(account_name, data_frame) {
  account_rep <- rep(account_name, nrow(data_frame))
  account_matrix <- data.frame(Account = account_rep, Username = data_frame$username)
  return(account_matrix)
}

```

edgelist for followers:

```

el_followers <- lapply(usernames, function(account_name) {
  followers_df <- get(paste0(account_name, "_followers_df"))
  create_edgelist(account_name, followers_df)
})
el_followers <- as.matrix(do.call(rbind, el_followers))

```

edgelist for friends:

```
el_followings <- lapply(usernames, function(account_name) {  
  following_df <- get(paste0(account_name, "_followings_df"))  
  create_edgelist(account_name, following_df)  
})  
el_followings <- as.matrix(do.call(rbind, el_followings))
```

combined edgelist for followers and friends:

```
el <- rbind(  
  el_followings,  
  el_followers  
)
```

Let's check the dimension and class of our final edgelist:

```
dim(el)
```

```
## [1] 446  2
```

```
class(el)
```

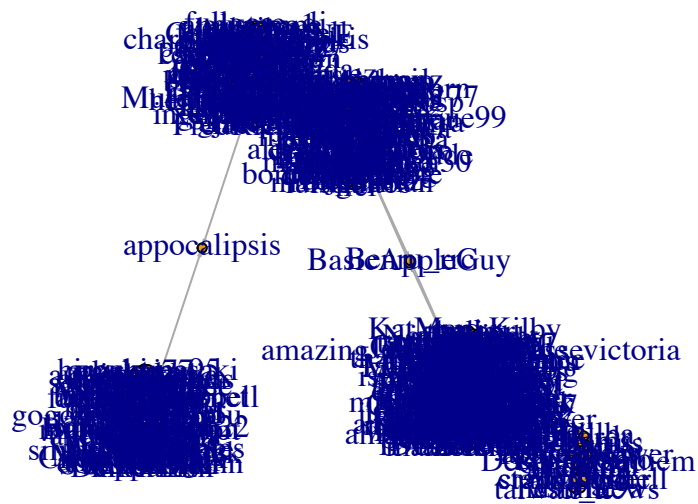
```
## [1] "matrix" "array"
```

Now that we have the edgelist needed for the graph, we can plot our graph:

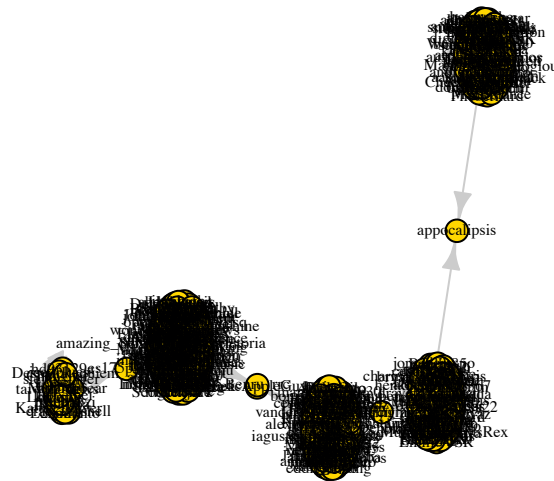
5.iv.ii Plotting the Graph:

We make use of 'graph_from_edgelist()' function under igraph library to plot the graph for our data.

```
g=graph_from_edgelist(el, directed = TRUE)  
plot(g, edge.arrow.size = 0.2, vertex.size = 4, vertex.label.cex = 0.7)
```

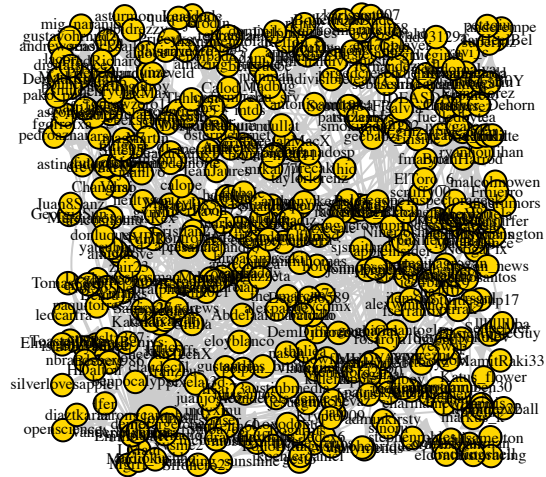



```
plot(g, layout = layout.fruchterman.reingold, vertex.size = 10,
edge.arrow.size=0.7, vertex.color="gold",
vertex.label.color="black",vertex.label.cex=0.5,edge.color="gray80")
```



random layout:

```
plot(g, layout = layout.random, vertex.size = 10, edge.arrow.size=0.5,
vertex.color="gold",
vertex.label.color="black",vertex.label.cex=0.5,edge.color="gray80")
```



These plots are not very readable, so we can take a subgraph and plot this one instead. But it is not the main graph we aimed to have.

```
g2 = induced_subgraph(g, which(degree(g, mode = "all") > 1))
plot(g2, layout = layout.fruchterman.reingold, vertex.size = 15, edge.arrow.size=0.8,
vertex.color="pink", vertex.label.color="black")
```



6. Question 6

6. Find the most central users in your graph using all centrality measures you learned in this subject. Comment on your findings.

We have mainly studied three centrality measures. They are:

Degree Centrality Closeness Centrality Betweenness Centrality

6.i Degree centrality

Degree centrality is a network centrality measure that quantifies the number of direct connections or edges a node has within a network. Its main feature is that it identifies how well-connected or popular a node is by counting its immediate neighbors, making it a straightforward measure to determine a node's local importance in a network.

```
sort(degree(g), decreasing=TRUE)[1:15]
```

##	AppleFanboy	Applelianos	appleinsider	applesfera
##	155	97	89	86
##	appleabear	Juan8Sanz	benpocalypse	michaldivismusic
##	20	3	2	2
##	technotramp	ryanhoulihan	luckie_reubs	drewzero1

```
##          2          2          2          2
##      br00t4c      Ninerjoshua      cwilliams25
##          2          2          2
```

```
centralIDs=order(degree(g), decreasing=TRUE)[1:7]
V(g)[centralIDs]
```

```
## + 7/418 vertices, named, from f63512c:
## [1] AppleFanboy Applelianos appleinsider applesfera appleabear
## [6] Juan8Sanz benpocalypse
```

```
degree(g)[centralIDs]
```

```
## AppleFanboy Applelianos appleinsider applesfera appleabear Juan8Sanz
##          155          97          89          86          20          3
## benpocalypse
##          2
```

Thus according to the degree centrality AppleFanboy, Applelianos, appleinsider, applesfera, appleabear, Juan8Sanz, and benpocalypse are the most central users.

6.ii Closeness Centrality

Closeness Centrality is a network centrality measure that quantifies how quickly a node can interact with other nodes in a network. It calculates the reciprocal of the average distance from a node to all other nodes. The main feature of closeness centrality is that it identifies nodes that can efficiently spread information or influence to other parts of the network, emphasizing their role in facilitating quick communication and interaction.

```
sort(closeness(g), decreasing=TRUE)[1:5]
```

```
## appleabear applesfera appleinsider AppleFanboy Applelianos
## 0.050000000 0.012048193 0.011363636 0.007575758 0.003846154
```

```
centralIDs=order(closeness(g), decreasing=TRUE)[1:5]
V(g)[centralIDs]
```

```
## + 5/418 vertices, named, from f63512c:
## [1] appleabear applesfera appleinsider AppleFanboy Applelianos
```

```
closeness(g)[centralIDs]
```

```
## appleabear applesfera appleinsider AppleFanboy Applelianos
## 0.050000000 0.012048193 0.011363636 0.007575758 0.003846154
```

Thus according to the Closeness centrality appleabear, applesfera, appleinsider, AppleFanboy and Applelianos are the most central users.

6.iii Betweenness centrality

Betweenness Centrality is a network centrality measure that identifies nodes that act as bridges or intermediaries between other nodes in a network. It quantifies the number of shortest paths passing through a node, highlighting its role in connecting different parts of the network. The main feature of betweenness centrality is its ability to reveal nodes that control the flow of information or resources, making it crucial for maintaining network cohesion and communication.

```
sort(betweenness(g), decreasing=TRUE)[1:5]
```

```
## applesfera appleinsider flargh WGallagher rsgnl
##          82           0           0           0           0
```

```
centralIDs=order(betweenness(g), decreasing=TRUE)[1:5]
V(g)[centralIDs]
```

```
## + 5/418 vertices, named, from f63512c:
## [1] applesfera appleinsider flargh WGallagher rsgnl
```

Thus according to the betweenness centrality applesfera seems the most central user.

7. References

Libretexts (2023) 9.4: Distribution needed for hypothesis testing, Statistics LibreTexts. Available at: https://stats.libretexts.org/Courses/Los_Angeles_City_College/Introductory_Statistics/09%3A_Hypothesis_Testing_with_One_Sample/9.04%3A_Distribution_Needed_for_Hypothesis_Testing (Accessed: 03 October 2023).

Hootsuite. (2022). What Is Mastodon and Why Is Everyone Going There? Hootsuite. <https://blog.hootsuite.com/what-is-mastodon/> (Accessed: 08 October 2023)